What eACBN0.f90 actually does

The ACBN0 kernels inside hp.x, and what they re-implement from PW/src/new_ns_v.f90

Code report • 14 September 2026

This is a description of the code, not of the validation. It answers two questions: what the kernels compute, and which parts of them are transplanted from the fork versus written here. Method reference: S. Lee and Y.-W. Son, *Phys. Rev. Research* **2**, 043410 (2020); ACBN0 itself: Agapito, Curtarolo and Buongiorno Nardelli, *Phys. Rev. X* **5**, 011006 (2015).

1 The quantity being computed

ACBN0 is not linear response. It never perturbs anything. It evaluates a *ratio* on a converged ground state: the Coulomb (and exchange) integral over the Hubbard manifold, weighted by *renormalised* occupations, divided by the same integral weighted by *bare* occupations. The numerator is screened by how much of each Bloch state actually lives on the manifold; the denominator is not. Their ratio is the effective U.

1.1 The working formulae, as the code has them

Write $P^{\sigma}_{mm'}$ for the renormalised on-site occupation matrix, n^{σ}_{mm} for the bare one, and $(m_3 m_1|m_4 m_2)$ for a two-electron repulsion integral over real spherical harmonics. Then for a manifold that is not a bare s shell:

$$A_0^U = \sum_{m_1 m_2 m_3 m_4} \left[P^{\uparrow}_{m_3 m_1} P^{\uparrow}_{m_4 m_2} + P^{\downarrow}_{m_3 m_1} P^{\downarrow}_{m_4 m_2} + P^{\uparrow}_{m_3 m_1} P^{\downarrow}_{m_4 m_2} + P^{\downarrow}_{m_3 m_1} P^{\uparrow}_{m_4 m_2} \right] (m_3 m_1|m_4 m_2) \tag{1}$$

$$A_0^J = \sum_{m_1 m_2 m_3 m_4} \left[P^{\uparrow}_{m_3 m_1} P^{\uparrow}_{m_4 m_2} + P^{\downarrow}_{m_3 m_1} P^{\downarrow}_{m_4 m_2} \right] (m_3 m_2|m_4 m_1) \tag{2}$$

and the denominators, built from bare occupations only,

$$\mathrm{HF}^U = \sum_{m_1 \neq m_2} \left[n^{\uparrow}_{m_1} n^{\uparrow}_{m_2} + n^{\downarrow}_{m_1} n^{\downarrow}_{m_2} + n^{\uparrow}_{m_1} n^{\downarrow}_{m_2} + n^{\downarrow}_{m_1} n^{\uparrow}_{m_2} \right] + \sum_{m_1 = m_2} \left[n^{\uparrow}_{m_1} n^{\downarrow}_{m_2} + n^{\downarrow}_{m_1} n^{\uparrow}_{m_2} \right] \tag{3}$$

$$\mathrm{HF}^J = \sum_{m_1 \neq m_2} \left[n^{\uparrow}_{m_1} n^{\uparrow}_{m_2} + n^{\downarrow}_{m_1} n^{\downarrow}_{m_2} \right] \tag{4}$$

giving

$$\boxed{U = \frac{A_0^U}{\mathrm{HF}^U} - \frac{A_0^J}{\mathrm{HF}^J}} \qquad \text{and, for a pure } s \text{ manifold,} \qquad U = \frac{A_0^U}{\mathrm{HF}^U}.$$

Two structural points fall straight out of these expressions.

- The s case drops the exchange term *and* the same-spin part of the direct term. A single s orbital has no orbital exchange partner, and the same-spin Hartree contribution is cancelled by Pauli exclusion, so only the $\uparrow\downarrow$ products survive. In the code this is the IF (lch(m1,nt) == 1 .AND. lch(m2,nt) == 1) branch — and note it tests *both* indices, because with background channels m_1 and m_2 can sit in different ℓ.
- The exchange integral has its indices transposed relative to the direct one: $(m_3 m_1|m_4 m_2)$ against $(m_3 m_2|m_4 m_1)$. That single swap is the whole difference between A_0^U and A_0^J in the source.

1.2 The inter-site term

For V the same construction runs over *pairs* of atoms (I, J) in neighbouring cells. The occupation matrix gains three indices — ns(m1,m2,spin,na1,na2,nnc) instead of ns(m1,m2,spin,na) — and the two-electron integral is evaluated with its two charge centres at τ_I and $\tau_J + \mathbf{R}$ rather than both at τ_I. The formula is simpler than the on-site one:

$$V_{IJ} \;=\; \frac{1}{2}\,\frac{E_0^V(I,J)}{\mathrm{HF}^V(I,J)}.$$

No exchange term, and a factor $\frac{1}{2}$ because the pair is accumulated twice, once as (I, J) and once as (J, I).

2 How the $+U$ kernel runs

hp_acbn0_ns_a0 is about 400 lines. Its shape:

1. **Project.** For each k-point, read the KS wavefunctions (iunwfc) and the ortho-atomic Hubbard projectors (iunhub), and form $\langle\varphi_m|\psi_{n\mathbf{k}}\rangle$ with calbec. This is the only place the wavefunctions are touched.
2. **Bare occupations n.** Accumulate $n^{\sigma}_{m_1 m_2} = \sum_{n\mathbf{k}} w_{n\mathbf{k}}\,\langle\psi|\varphi_{m_2}\rangle\langle\varphi_{m_1}|\psi\rangle$. Standard DFT+U occupation matrix.
3. **Per-band manifold weight.** For every band separately, accumulate $|\langle\varphi_m|\psi_{n\mathbf{k}}\rangle|^2$ into num(ibnd)%O, indexed by ℓ and by *type* rather than by atom, then sum it over all atoms of the same species (upf(nt1)%psd == upf(nt2)%psd). This is the ACBN0 renormalisation factor: how much of band $n\mathbf{k}$ sits on that species' manifold.
4. **Renormalised occupations P.** Repeat step 2, but weight each band by the factor from step 3. That is the single substantive difference between n and P, and the entire physical content of ACBN0.
5. **Symmetrise.** Both n and P go through hp_acbn0_symm_n4.
6. **Assemble.** Build A_0 from P and HF from n per Eqs. (1)–(4), then take the ratio.

2.1 An observation about the renormalisation

Step 3 fills *only* the diagonal num(ibnd)%O(l,l,...). Step 4 then reads num(ibnd)%O(l1,l2,...) with ℓ_1 from m_1 and ℓ_2 from m_2. So whenever m_1 and m_2 lie in *different* channels, the renormalisation weight is zero and the corresponding block of P vanishes by construction.

For every single-channel system this is invisible: $\ell_1 = \ell_2$ always. It only bites on a multi-channel manifold such as GaAs (Ga $4s + 4p$), where it means the s–p cross-block of P is identically zero while the s–p block of the *bare* occupations is not. **This is the original code's behaviour and we reproduce it exactly** — which is precisely why our GaAs run matches pw.x across all 204 lines. I am flagging it as a property of the method as implemented, not as a defect, because I have not checked it against the ACBN0 paper's intent and it is the kind of thing worth confirming with the author before anything is published on multi-channel manifolds.

2.2 The two-electron integrals

The integrals come from outside this file. pao_3g returns a three-Gaussian fit to the pseudo-atomic orbital for a given species and ℓ, and compute_eri evaluates $(ab|cd)$ analytically over those Gaussians at four given centres. hp_acbn0_orberi is the small bookkeeping routine that walks the real-spherical-harmonic index: 1 for s; 2, 3, 4 for p; 5 . . . 9 for d. It is a stateful cycler — called once per m inside each loop, it advances its argument, so the m-th orbital of the manifold gets the m-th harmonic index.

This is also where the s, p, d-only restriction comes from: pao_3g has no f-shell fit and returns zero coefficients *silently*, which would make every integral vanish and U come out as $0/0$. Both check routines refuse $\ell > 2$ explicitly for that reason.

3 Provenance: what is transplanted, what is new

3.1 Transplanted from PW/src/new_ns_v.f90

Our routine	From	Stmts	Role
hp_acbn0_ns_a0	new_ns_a0 233–635	176	the $+U$ kernel
hp_acbn0_ns_e0_1	new_ns_e0_1 636–1808	577	the $+U+V$ kernel
hp_acbn0_ns_v	new_ns_v 10–232	103	occupation matrices alone
hp_acbn0_orberi	orberi 3780–3819	25	harmonic index cycler
hp_acbn0_symm_n4	symmetrize_n4 2964–3167	86	symmetrise $n(m_1, m_2, \sigma, I)$
hp_acbn0_symm_n4m	symmetrize_n4m 3168–3399	104	ditto, magnetic
hp_acbn0_symm_n5	symmetrize_n5 3400–3779	174	symmetrise the 6-index pair matrix
total transplanted		**1245**	

How faithful. The transplanted bodies were diffed against the fork by a script (normdiff4.py) that joins Fortran continuations into whole statements, strips comments, keeps only executable code, collapses the mechanical renames to common tokens, and canonicalises the intended systematic edits. **1245 statements, 18 differing**, in three classes:

- 6 — hp_acbn0_orberi's dummy argument renamed ehub_l → lp1 (cosmetic; it is a dummy name).
- 8 — the dropped stop_acbn0 convergence short-circuit and its ehub_v_diff bookkeeping. hp.x is one-shot and has no SCF history, so there is nothing to short-circuit.
- 4 — the two bug-fix guards (§4).

hp_acbn0_ns_a0 and hp_acbn0_ns_v come out at **zero** differences. The measurement is stronger than it looks: since stage V4 the canonicaliser no longer collapses the m index, so every ehub_l(m1,nt) in the fork must match lch(m1,nt) here literally, index by index — all 225 of them do.

3.2 Written here

MODULE hp_acbn0_nn	The neighbour table. Reproduces hubbard_nns() and the nnnc count from input.f90:2144, but built from tau/at/alat and a cutoff rather than read from ldaV. Also hp_acbn0_symmetry and hp_acbn0_get_nnc, from PW/src/hubbard_nns.f90.
MODULE hp_acbn0_man	The manifold. Maps stock ldaU's ll(m,nt), ldim_u, ldmx_tot and the three projector blocks (offsetU/_back/_back1) onto the contiguous view lm/lch/ofs that the kernels are written against. Entirely new — ldaV stored the manifold contiguously, so the fork never needed it.
drivers	hp_acbn0, hp_acbn0_u, hp_acbn0_v: buffer handling, orthoUwfc, d_matrix, allocation, cleanup. Replaces what sum_band did for the fork.
guards	hp_acbn0_check, hp_acbn0_check_v: preconditions the kernels assume silently (ortho-atomic projectors, $n_{\mathrm{spin}} = 2$, $s/p/d$ only, no gamma_only). Each of these fails as a *wrong number* rather than a crash if unchecked.
output	hp_acbn0_write, _write_v, _print_uv, hp_acbn0_manlab. _print_uv deliberately reproduces pw.x's own table format so that every comparison is a plain diff.

3.3 Deliberately not transplanted

new_ns_e0_2 (acbn0_type = 3) and the entire noncollinear set — new_ns_v_nc, new_ns_e0_nc_1, symmetrize_n4_nc, _n4m_nc, _n5_nc, roughly 2300 further lines. This is why the authors' GaAs reference, which is noncollinear with spin–orbit, remains unreachable.

4 Four bugs found in the original

The fork's patch touches 60 files and **none is under** HP/ — ACBN0 was implemented in pw.x only. pw.x initialises MODULE ldaV from two paths (the input file, and the XML); hp.x only ever walks the second. Every bug below is a consequence.

Bug 1 — remove_ehub_u / _v undefined (silent). No initialiser in ldaV.f90; assigned only at input.f90:2083. The kernels multiply the final U by them *twice*. Undefined, this does not crash: it silently scales or zeroes every Hubbard parameter. Not hypothetical — the authors' own GaAs.acbn0.in_noncollienar sets remove_ehub_u(1,1) = 0.

Bug 2 — iunwfc closed at the insertion point (crash). read_file buffers the wavefunctions then closes the unit with close_buffer(...,'KEEP'). 'KEEP' preserves the *files*, but the call also deregisters the unit, so get_buffer falls through to davcio on a closed unit. Stock hp.x never trips this because DFPT reads through iuwfc, opened later.

Bug 3 — hubbard_get_nnc returns an undefined index. hubbard_nns.f90:241. nnc is assigned only inside IF (MAXVAL(...) == 1), so a cell carrying no neighbour pair falls through untouched. hubbard_symmetry then asks for the index of a *rotated* cell, which need not be occupied, and symmetrize_n5 indexes n(nnc) with whatever was on the stack. Silent in an optimised build whenever the garbage index lands in a mapped page; -check bounds,uninit named it at once.

Bug 4 — l_separation writes one element before the array (latent, still live). In symmetrize_n4, _n4m and _n5, a type with no Hubbard manifold has ehub_l(1,nt) = 0 and the next line writes l_separation(0,nt). Dormant when every type is Hubbard-active — true of NiO, MnO, ZnO, CdO, CrI_3 and GaAs — and it fires the moment a spectator species is added.

Bugs 3 and 4 are the two guards that show up in the static diff. Bugs 1 and 2 needed no guard: bug 1 disappears because ldaU has no such mask, and bug 2 is handled by reopening the unit in the driver.

5 Where to look in the file

Line	Routine	
1	file header	the ldaV → ldaU translation table
94	MODULE hp_acbn0_nn	neighbour table
491	MODULE hp_acbn0_man	manifold, and the wfcU column map
704	hp_acbn0_u	+U driver
790	hp_acbn0_check	+U preconditions
890	hp_acbn0_write	eACBN0_hub.dat
1031	hp_acbn0_ns_a0	**the +U kernel**
1436	hp_acbn0_symm_n4	symmetrisation
1648	hp_acbn0_orberi	harmonic index cycler
1701	hp_acbn0	dispatcher on the acbn0 tag
1721	hp_acbn0_v	+U + V driver
2215	hp_acbn0_ns_e0_1	**the +U + V kernel**
3372	hp_acbn0_ns_v	occupation matrices alone
3596	hp_acbn0_symm_n4m, 3839 _n5	symmetrisation

4234 lines, two modules, 22 subroutines and one function. Seven other files in HP/src/ carry small edits: hpcom.f90 (two variables), hp_readin.f90 (namelist, defaults, four relaxed DFPT guards), hp_bcast_input.f90 (two broadcasts), hp_main.f90 (the dispatch, placed above hp_init), hp_print_clock.f90, Makefile, make.depend.

6 Two things I would change

Stale guard exemptions in hp_readin.f90**.** Three guards are written .AND. acbn0 /= 2, which was correct at stage V2 when the *+V* branch still ran on ldaV. Since V3 it runs on ldaU and *does* need lda_plus_u, ortho-atomic projectors and lda_plus_u_kind ≠ 1. Not a correctness bug — all three are backstopped by hp_acbn0_check_v, so a bad input gets a clear error rather than a wrong number — but the exemptions are dead weight and the comments are two stages out of date.

remove_ehub_u **has no route in.** It is input-only and is not round-tripped through the XML, so we omit the factors entirely — exactly equivalent whenever the pw.x run used the documented default of 1, and wrong for any run that masks a channel, as the authors' GaAs input does. The fix is one &INPUTHP tag.

Companion documents: REPORT-eacbn0-hpx.pdf (validation), slides.html (group meeting), logs/260901-eacbn0-hpx.txt and logs/260904-eacbn0-v4.txt (session logs). The faithfulness measurement is re-runnable: python3 normdiff4.py.